

ملخص شامل لـ

• system design •

دليل تعليمي مختصر ومنظم

١٣ مارس ٢٠٢٦

جدول المحتويات

1. ما هو تصميم الأنظمة ولماذا هو مهم
2. المتطلبات الوظيفية وغير الوظيفية للنظام
3. مفاهيم قابلية التوسع (Scalability)
4. التوسع العمودي (Vertical Scaling) والأفقي (Horizontal Scaling)
5. الموثوقية وتوفر النظام (Reliability & Availability)
6. تحمل الأخطاء (Fault Tolerance)
7. التصميم المتجانس (Monolithic Architecture)
8. الخدمات المصغرة (Microservices Architecture)
9. قواعد البيانات والتخزين — الفهرسة (Indexing)
10. بروتوكولات الشبكات الأساسية (HTTP / TCP/IP)
11. نظم الرسائل (Message Queues)
12. موازنة الحمل (Load Balancing)
13. المصادقة والتفويض (Authentication & Authorization)
14. مقاييس أداء النظام (Latency / Throughput / RPS)
15. مراقبة الأنظمة والتسجيل (Monitoring & Logging)

ما هو تصميم الأنظمة ولماذا هو مهم

ما هو تصميم الأنظمة ولماذا هو مهم

تصميم الأنظمة هو فن وعلم بناء أنظمة برمجية قوية وفعالة. إنه يشبه الخريطة التفصيلية التي ترشد المهندسين المعماريين لبناء منزل كبير ومعقد. يساعدنا في التفكير في كيفية عمل الأجزاء المختلفة معًا لتشكيل حل برمجي متكامل.

المفهوم الأساسي: التخطيط للبناء البرمجي

تصميم الأنظمة يتضمن تحديد المكونات المختلفة للنظام، وكيف ستتفاعل مع بعضها البعض، وكيف ستدقق البيانات بينها. كما يهتم بمعالجة جوانب مثل قابلية التوسع (قدرة النظام على النمو)، الأمان، الأداء، وقابلية الصيانة. أهميته تكمن في تجنب الأخطاء المكلفة لاحقًا، وضمان أن النظام يعمل بكفاءة ويمكن تطويره وتوسيع نطاقه في المستقبل.

مثال توضيحي

```
# مثال على تصميم نظام بسيط لمنصة تدوين
system_name: BlogPlatform
components:
  - name: AuthService
    description: مسؤول عن تسجيل الدخول وإدارة المستخدمين.
    technology: REST API, PostgreSQL
  - name: PostService
    description: يدير إنشاء المقالات وتعديلها.
    technology: REST API, MongoDB
  - name: CommentService
    description: يعالج التعليقات على المقالات.
    technology: Message Queue, Redis
  - name: FrontendApp
    description: واجهة المستخدم الرسومية للمنصة.
    technology: React, Nginx
```

شرح المثال

يظهر هذا المثال البسيط كيف يمكننا تخطيط مكونات نظام تدوين. كل مكون (مثل خدمة المصادقة أو خدمة المقالات) له وصف واضح لوظيفته والتقنيات المقترحة. هذا التخطيط المبدئي يساعد على فهم النظام ككل، وتحديد العلاقات بين أجزائه، واختيار التقنيات المناسبة قبل البدء في كتابة أي سطر من التعليمات البرمجية الفعلية.

- **تصميم الأنظمة** هو عملية التخطيط الشامل لبناء أنظمة برمجية معقدة.
- يساعد على ضمان **استقرار النظام**، **قابليته للتوسع**، و**أدائه الجيد** منذ البداية.
- يتضمن تحديد المكونات، تفاعلاتها، تدفق البيانات، ومعالجة تحديات مثل **الأمان والأداء**.

المتطلبات الوظيفية وغير الوظيفية للنظام

المتطلبات الوظيفية وغير الوظيفية للنظام

عند الشروع في تصميم أي نظام برمجي، من الضروري فهم ما يحتاجه هذا النظام بالضبط. تُصنف هذه الاحتياجات عادةً إلى نوعين رئيسيين: المتطلبات الوظيفية والمتطلبات غير الوظيفية، وكلاهما يلعب دوراً حاسماً في بناء نظام فعال وموثوق.

فهم المتطلبات الوظيفية وغير الوظيفية

المتطلبات الوظيفية (Functional Requirements) تحدد "ماذا" يجب أن يفعله النظام. هي الوظائف والميزات المباشرة التي يقدمها للمستخدمين. على سبيل المثال: قدرة المستخدم على تسجيل الدخول، عرض المنتجات، أو إتمام عملية الشراء. هذه المتطلبات تصف الإجراءات المحددة التي يتوجب على النظام تنفيذها.

أما **المتطلبات غير الوظيفية (Non-Functional Requirements)** فتحدد "كيف" يجب أن يعمل النظام. هي معايير الجودة والأداء التي يجب أن يلتزم بها النظام. وتشمل جوانب مثل الأمان، السرعة (زمن الاستجابة)، قابلية التوسع، الموثوقية، وسهولة الاستخدام. هذه المتطلبات لا تحدد وظيفة معينة، بل تحدد جودة هذه الوظائف.

مثال توضيحي

المتطلبات الوظيفية لنظام متجر إلكتروني:

- تسجيل دخول المستخدمين
- عرض قائمة المنتجات المتاحة
- إضافة المنتجات إلى سلة التسوق
- إتمام عملية الشراء والدفع

المتطلبات غير الوظيفية لنظام متجر إلكتروني:

- الأداء: زمن استجابة لا يتجاوز 2 ثانية في 90% من الطلبات
- الأمان: تشفير جميع بيانات المستخدمين الحساسة (مثل معلومات الدفع)
- %التوفر: ضمان وقت تشغيل للنظام بنسبة 99.9
- قابلية التوسع: دعم 100,000 مستخدم متزامن على الأقل

شرح المثال

يوضح المثال أعلاه كيف أن المتطلبات الوظيفية (مثل تسجيل الدخول وإتمام الشراء) تصف الوظائف الأساسية التي يتوقع المستخدمون الحصول عليها من المتجر الإلكتروني. في المقابل، تُحدد المتطلبات غير الوظيفية (مثل زمن الاستجابة والأمان) كيف ستُقدم هذه الوظائف بكفاءة وموثوقية، مما يضمن تجربة مستخدم جيدة ونظاماً آمناً ومستقراً. فهم كلاهما ضروري لتصميم نظام متكامل وناجح.

- المتطلبات الوظيفية تحدد الميزات الأساسية والوظائف المباشرة للنظام.
- المتطلبات غير الوظيفية تحدد معايير الجودة والأداء التي يجب أن يفي بها النظام.
- يجب تحديد وتقييم كلا النوعين من المتطلبات بعناية لضمان نجاح تصميم النظام وتنفيذه.

قابلية التوسع والموثوقية — مفاهيم قابلية التوسع (Scalability)

مفاهيم قابلية التوسع (Scalability)

في عالم تطوير الأنظمة، يُعد فهم قابلية التوسع أمرًا بالغ الأهمية. إنها القدرة على التعامل مع النمو وزيادة الطلب دون المساس بالأداء أو تجربة المستخدم.

ما هي قابلية التوسع؟

قابلية التوسع (Scalability) هي سمة لنظام أو شبكة أو عملية، تُعبر عن قدرتها على التعامل مع كمية متزايدة من العمل أو النمو المستقبلي بكفاءة. ببساطة، كيف يمكن لنظامك أن يتعامل مع عدد أكبر من المستخدمين أو المزيد من البيانات دون أن ينهار أو يصبح بطيئًا.

Code Example

```
#!/bin/bash

echo "-----"
echo "(Vertical Scaling):"
echo ".ترقية خادم واحد ليكون أقوى"
CURRENT_RAM="8GB"
CURRENT_CPU="4 Cores"
echo "الخادم الحالي: RAM $CURRENT_RAM, CPU $CURRENT_CPU"
echo "..."
UPGRADED_RAM="32GB"
UPGRADED_CPU="16 Cores"
echo "بعد الترقية: RAM $UPGRADED_RAM, CPU $UPGRADED_CPU"
echo "-----"

echo "(Horizontal Scaling):"
echo ".إضافة المزيد من الخوادم للتعامل مع الطلب المتزايد"
INITIAL_SERVERS=2
echo "عدد الخوادم الأولية: $INITIAL_SERVERS"
echo "..."
```



```
ADDED_SERVERS=3
TOTAL_SERVERS=$((INITIAL_SERVERS + ADDED_SERVERS))
echo "خوادم جديدة $ADDED_SERVERS تمت إضافة"
echo "$TOTAL_SERVERS : العدد الإجمالي للخوادم الآن"
echo "كل خادم يمكنه معالجة جزء من العمل"
echo "-----"
```

Explanation

يوضح هذا المثال المفاهيمي نوعين رئيسيين من التوسع. الجزء الأول يمثل **التوسع الرأسى (Vertical Scaling)**، حيث نقوم بترقية خادم واحد بإضافة المزيد من الموارد (مثل الذاكرة والمعالج) لجعله أقوى. أما الجزء الثاني فيمثل **التوسع الأفقى (Horizontal Scaling)**، حيث نقوم بإضافة المزيد من الخوادم المتطابقة لتوزيع الحمل عليها، مما يسمح للنظام بمعالجة المزيد من الطلبات الإجمالية.

- قابلية التوسع هي قدرة النظام على التعامل مع زيادة العمل أو المستخدمين.
- **التوسع الرأسى (Vertical Scaling)**: يتم بتحسين موارد خادم واحد (أقوى).
- **التوسع الأفقى (Horizontal Scaling)**: يتم بإضافة المزيد من الخوادم لتوزيع الحمل (أكثر).

قابلية التوسع والموثوقية — التوسع العمودي (Vertical Scaling) والأفقي (Horizontal Scaling)

التوسع العمودي (Vertical Scaling) والأفقي (Horizontal Scaling)

في عالم تطوير الأنظمة، قابلية التوسع تعني قدرة النظام على التعامل مع زيادة الحمل (مثل عدد المستخدمين أو كمية البيانات). الموثوقية هي ضمان استمرار عمل النظام بشكل صحيح حتى مع وجود أعطال. التوسع العمودي والأفقي هما استراتيجيتان أساسيتان لتحقيق ذلك.

مفهوم التوسع العمودي (Vertical Scaling)

التوسع العمودي، المعروف أيضًا باسم "التحجيم لأعلى" (Scale Up)، يعني زيادة موارد خادم واحد (مثل وحدة المعالجة المركزية CPU، الذاكرة RAM، أو مساحة التخزين) لجعله أقوى وأكثر قدرة على التعامل مع الأعباء المتزايدة. فكر في الأمر كترقية جهاز الكمبيوتر الخاص بك بمكونات أفضل.

مفهوم التوسع الأفقي (Horizontal Scaling)

التوسع الأفقي، المعروف أيضًا باسم "التحجيم للخارج" (Scale Out)، يعني إضافة المزيد من الخوادم إلى جهازك. بدلاً من جعل خادم واحد أقوى، تقوم بتوزيع العمل على عدة خوادم تعمل معًا. هذا يوفر مرونة أكبر ومقاومة للأعطال.

Code Example

```
# (Vertical Scaling) مثال مفاهيمي للتوسع العمودي
echo "...ترقية خادم واحد بزيادة الذاكرة ووحدة المعالجة المركزية"
echo "RAM إضافة 64 جيجابايت -"
echo "ترقية وحدة المعالجة المركزية إلى 16 نواة -"
echo "...الخادم الآن أقوى ولكنه لا يزال نقطة فشل واحدة"

echo ""
```

(Horizontal Scaling) مثال مفاهيمي للتوسع الأفقي

echo "...إضافة خوادم متعددة لتوزيع الحمل"

echo "نشر نسخة جديدة من التطبيق على الخادم رقم 2 - "

echo "نشر نسخة جديدة من التطبيق على الخادم رقم 3 - "

echo ".الآن لدينا 3 خوادم تعمل معًا، مما يزيد من الموثوقية والقدرة"

شرح الأمثلة

توضح الأمثلة المفاهيمية كيف يمكن التفكير في التوسع. في التوسع العمودي، نحن نركز على جعل خادم واحد أكثر قوة. هذا له حدوده؛ ففي النهاية لا يمكن ترقية خادم واحد إلى ما لا نهاية. أما في التوسع الأفقي، فنحن نضيف المزيد من الخوادم، مما يسمح بتوزيع الحمل ويوفر مرونة أكبر ومقاومة ضد توقف أحد الخوادم عن العمل. يتم عادة استخدام موازن حمل (Load Balancer) لتوزيع الطلبات بين هذه الخوادم المتعددة.

- **التوسع العمودي:** تقوية خادم واحد بزيادة موارده (CPU، RAM، تخزين). أسهل في الإدارة لكن له حدود.
- **التوسع الأفقي:** إضافة المزيد من الخوادم لتوزيع الحمل. يوفر مرونة وموثوقية أعلى ولكنه أكثر تعقيدًا.
- لأنظمة الإنتاج الكبيرة، غالبًا ما يكون الجمع بين الاستراتيجيتين هو الحل الأمثل لتحقيق الأداء العالي والموثوقية.

قابلية التوسع والموثوقية — الموثوقية وتوفر النظام (Reliability & Availability)

الموثوقية وتوفر النظام (Reliability & Availability)

مرحباً بك في درس اليوم حول الموثوقية وتوفر النظام، وهما مفهومان أساسيان في تصميم أي نظام برمجي ناجح. ففهمهما يساعدنا على بناء أنظمة تعمل باستمرار وبدون انقطاع وتقدم تجربة ممتازة للمستخدمين.

المفهوم الأساسي

الموثوقية (Reliability) تعني قدرة النظام على أداء وظائفه المحددة بشكل صحيح ومستمر لفترة زمنية معينة دون فشل. إنها تركز على تقليل الأخطاء ومنع الأعطال. بينما **توفر النظام (Availability)** يشير إلى النسبة المئوية للوقت الذي يكون فيه النظام قيد التشغيل ومتاحًا للاستخدام من قبل المستخدمين. نظام موثوق قد يتعطل، لكنه مصمم ليستعيد نفسه بسرعة. نظام متوفر هو النظام الذي يمكن الوصول إليه بشكل عام عندما يحتاجه المستخدمون.

مثال توضيحي (فحص توفر الخدمة)

```
import time

def check_service_status(service_name):
    """
    يحاكي فحص حالة خدمة معينة.
    قاعدة بيانات، أو فحص الموارد، API عادةً ما يتضمن هذا الاتصال بـ
    """
    print(f"Checking status for {service_name}...")
    # Simulate a potential failure or delay
    if service_name == "DatabaseService" and time.time() % 5 < 2:
        return False # Simulate temporary unavailability
    return True

def monitor_system_availability(services):
```

```

"""
يراقب توفر مجموعة من الخدمات ويطلع حالتها.
"""

print("\n--- System Availability Monitor ---")
all_available = True
for service in services:
    if check_service_status(service):
        print(f"✅ {service}: Available")
    else:
        print(f"❌ {service}: Unavailable")
        all_available = False
if all_available:
    print("🟢 All core services are available.")
else:
    print("🔴 Some services are currently unavailable. Investigate!")

# مثال على الاستخدام:
core_services = ["WebServer", "APIGateway", "DatabaseService", "CacheService"]
monitor_system_availability(core_services)

```

شرح المثال

يوضح هذا المثال المبسّط كيفية مراقبة توفر الخدمات في نظام ما. الدالة `check_service_status` تحاكي فحصًا بسيطًا لخدمة معينة (مثل فحص قاعدة بيانات أو خادم ويب). في الأنظمة الحقيقية، ستتصل هذه الدالة بنقاط نهاية صحية (health endpoints) أو تقوم بفحص موارد محددة. الدالة `monitor_system_availability` تقوم بفحص عدة خدمات أساسية وتحديد ما إذا كانت جميعها متاحة، مما يعطينا مؤشرًا مباشرًا على توفر النظام ككل. هذا النوع من المراقبة ضروري لضمان أن النظام يعمل بشكل مستمر ويمكن للمستخدمين الوصول إليه.

- **الموثوقية** هي قدرة النظام على العمل بشكل صحيح دون فشل متكرر.
- **توفر النظام** هو النسبة المئوية للوقت الذي يكون فيه النظام متاحًا للاستخدام.
- تصميم أنظمة مقاومة للأخطاء واستخدام آليات المراقبة المستمرة يعزز من الموثوقية والتوفر.

قابلية التوسع والموثوقية — تحمل الأخطاء (Fault Tolerance)

تحمل الأخطاء (Fault Tolerance)

في عالم الأنظمة المعقدة، يُعدّ الفشل أمرًا واريًا. تحمل الأخطاء هو قدرة النظام على الاستمرار في العمل بشكل طبيعي أو شبه طبيعي حتى عند تعطل أجزاء منه. إنه مبدأ أساسي لبناء أنظمة موثوقة وعالية التوافر.

المفهوم الأساسي

تحمل الأخطاء هو تصميم نظام بحيث يمكنه التعامل مع فشل مكون واحد أو أكثر دون توقف الخدمة بالكامل. يتم تحقيق ذلك عادةً من خلال التكرار (**Redundancy**)، حيث يتم توفير نسخ احتياطية للمكونات، وآليات مثل الفشل المتجاوز (**Failover**) التي تنقل العمل تلقائيًا إلى هذه النسخ الاحتياطية عند اكتشاف فشل.

مثال برمجي

```
import time
import random

def connect_to_service():
    # تحاكي خدمة قد تفشل أحياناً
    if random.random() < 0.6: # فرصة للفشل 60%
        raise ConnectionError("فشل الاتصال بالخدمة")
    return "تم الاتصال بنجاح"

def reliable_service_call(max_retries=3):
    for attempt in range(max_retries):
        try:
            print(f"المحاولة {attempt + 1} محاولة الاتصال...")
            result = connect_to_service()
            print(result)
            return result
```

```
except ConnectionError as e:
    print(f"...إعادة المحاولة بعد 1 ثانية. {e}: حدث خطأ")
    time.sleep(1)
print("فشلت جميع المحاولات. تعذر الاتصال بالخدمة")
return None

# مثال على الاستخدام
# reliable_service_call()
```

شرح المثال

يوضح هذا المثال مفهوم تحمل الأخطاء البسيط باستخدام آلية إعادة المحاولة (**Retries**). الدالة `connect_to_service` تحاكي اتصالاً بخدمة خارجية قد يفشل بنسبة 60%. الدالة `reliable_service_call` تحاول الاتصال بالخدمة عدة مرات (3 مرات في هذا المثال). إذا فشلت محاولة، فإنها تنتظر قليلاً ثم تعيد المحاولة، مما يزيد من احتمالية نجاح الاتصال حتى لو كان هناك فشل عابر.

- تحمل الأخطاء يضمن استمرارية الخدمة حتى عند فشل المكونات الفردية.
- يعتمد على مبادئ مثل التكرار، الفشل المتجاوز، وآليات إعادة المحاولة.
- يُعد ضروريًا لبناء أنظمة قوية وموثوقة تتطلب توافراً عالياً.

التصميم المتجانس (Monolithic Architecture)

التصميم المتجانس (Monolithic Architecture)

عند بناء أنظمة برمجية، نحتاج لاختيار طريقة تنظيم مكوناتها. التصميم المتجانس هو أحد هذه الأنماط، حيث يتم بناء التطبيق ككتلة واحدة مترابطة وتعمل كوحدة واحدة.

المفهوم الأساسي

التصميم المتجانس (Monolithic Architecture) هو نمط تصميم معماري يتم فيه بناء جميع مكونات التطبيق كوحدة واحدة غير قابلة للتجزئة. جميع وظائف التطبيق، مثل واجهة المستخدم، ومنطق العمل، وقاعدة البيانات، تتواجد ضمن قاعدة كود واحدة وتعمل كعملية واحدة.

شرح مبسط

في التصميم المتجانس، يكون كل شيء مرتبطًا ومدمجًا. هذا يجعل التطوير الأولي أسهل وأسرع، خاصة للمشاريع الصغيرة والفرق الصغيرة. التوزيع والنشر يكونان أيضًا أبسط لأنك تتعامل مع تطبيق واحد. ومع ذلك، كلما نما التطبيق وزادت تعقيداته، يصبح من الصعب صيانتها، وتوسيعها، وتحديث أجزاء منه بشكل مستقل، لأن أي تغيير صغير قد يتطلب إعادة نشر التطبيق بأكمله.

مثال برمجي

```
from flask import Flask, jsonify, request

app = Flask(__name__)

# User Management Module (conceptual)
@app.route('/users', methods=['GET'])
def get_users():
    # Logic to fetch all users
    return jsonify({"message": "Getting all users from a monolithic app."})

@app.route('/users', methods=['POST'])
def create_user():
```



```

# Logic to create a user
data = request.json
return jsonify({"message": f"Creating user: {data.get('name')}"})

# Product Management Module (conceptual)
@app.route('/products', methods=['GET'])
def get_products():
    # Logic to fetch all products
    return jsonify({"message": "Getting all products from a monolithic app."})

@app.route('/products/<int:product_id>', methods=['GET'])
def get_product(product_id):
    # Logic to fetch a specific product
    return jsonify({"message": f"Getting product with ID: {product_id}"})

if __name__ == '__main__':
    app.run(debug=True)

```

شرح المثال

يوضح هذا المثال البسيط لتطبيق Flask كيف يمكن تجميع وظائف مختلفة (مثل إدارة المستخدمين وإدارة المنتجات) ضمن ملف تطبيق واحد. في التصميم المتجانس، حتى لو كانت هذه الوظائف منفصلة منطقياً، فإنها تعيش وتعمل كجزء من عملية واحدة كبيرة. هذا يعني أن كل شيء يدور في نفس "الخدمة" أو "البرنامج" الكبير، مما يمثل جوهر التصميم المتجانس.

- جميع مكونات التطبيق (واجهة المستخدم، منطق العمل، قاعدة البيانات) تتجمع في وحدة برمجية واحدة كبيرة.
- يسهل البدء به وتطويره للمشاريع الصغيرة أو الفرق المحدودة في البداية.
- يصبح صعب التوسع، الصيانة، وتطبيق التحديثات المستقلة كلما كبر وتعقد النظام.

الخدمات المصغرة (Microservices Architecture)

الخدمات المصغرة (Microservices Architecture)

تُعد الخدمات المصغرة (Microservices) نمطًا معماريًا حديثًا وشائعًا لتطوير التطبيقات المعقدة، حيث يتم تقسيم التطبيق إلى مجموعة من الخدمات الصغيرة والمستقلة التي تعمل معًا.

المفهوم الأساسي

في هذا النمط، يتم بناء كل خدمة بشكل مستقل تمامًا، وتمتلك قاعدة بياناتها الخاصة، وتنفذ وظيفة عمل محددة وواحدة. تتواصل هذه الخدمات مع بعضها البعض عبر واجهات برمجة التطبيقات (APIs)، غالبًا باستخدام بروتوكولات خفيفة الوزن مثل HTTP/REST.

مثال توضيحي

```
# users_service.py
from flask import Flask, jsonify

app = Flask(__name__)

@app.route('/users/<int:user_id>', methods=['GET'])
def get_user(user_id):
    # Imagine fetching user data from a database specific to this service
    if user_id == 1:
        return jsonify({"id": 1, "name": "أحمد", "email": "ahmed@example.com"})
    return jsonify({"message": "المستخدم غير موجود"}), 404

if __name__ == '__main__':
    app.run(port=5001)
```

شرح المثال

يمثل هذا الكود خدمة مصغرة بسيطة باستخدام إطار عمل Flask في بايثون. هذه الخدمة مسؤولة فقط عن إدارة بيانات المستخدمين. عندما يتم طلب مسار `<users/<user_id/>`، تقوم الخدمة

بالاستجابة بمعلومات المستخدم المطلوبة أو رسالة خطأ إذا لم يتم العثور عليه. لاحظ كيف أن هذه الخدمة تركز على وظيفة واحدة محددة وهي "المستخدمون" فقط.

- كل خدمة مستقلة بذاتها وتؤدي وظيفة عمل محددة.
- تتواصل الخدمات عبر واجهات برمجة التطبيقات (APIs).
- يمكن نشر وتطوير وتوسيع نطاق كل خدمة بشكل منفصل عن الأخرى.

قواعد البيانات والتخزين — الفهرسة (Indexing)

الفهرسة (Indexing)

الفهرسة هي تقنية أساسية في قواعد البيانات تساعد على تسريع استرجاع البيانات. تخيل كتاباً ضخماً بدون فهرس؛ سيكون من الصعب جداً العثور على معلومات محددة. الفهرس في قواعد البيانات يعمل بنفس الطريقة تماماً.

ما هي الفهرسة؟

الفهرسة هي بنية بيانات خاصة تخزن نسخة من حقول معينة من جدول، مرتبة بطريقة تسهل البحث السريع. عند إنشاء فهرس على عمود أو مجموعة أعمدة، تقوم قاعدة البيانات بإنشاء هيكل يمكنها من الانتقال مباشرة إلى الصفوف المطلوبة بدلاً من مسح الجدول بأكمله. هذا يقلل بشكل كبير من وقت الاستعلام، خاصة في الجداول الكبيرة.

مثال عملي

```
-- إنشاء جدول بسيط للمستخدمين
CREATE TABLE Users (
    UserID INT PRIMARY KEY,
    UserName VARCHAR(100),
    Email VARCHAR(100)
);

-- إدخال بعض البيانات
INSERT INTO Users (UserID, UserName, Email) VALUES
(1, 'علي أحمد', 'ali.ahmed@example.com'),
(2, 'فاطمة محمود', 'fatma.mahmoud@example.com'),
(3, 'سعيد ناصر', 'saeed.nasser@example.com');

-- إنشاء فهرس على عمود البريد الإلكتروني لتسريع البحث
CREATE INDEX idx_email ON Users (Email);

-- استعلام سيستفيد من الفهرس
SELECT * FROM Users WHERE Email = 'fatma.mahmoud@example.com';
```

شرح المثال

في هذا المثال، قمنا أولاً بإنشاء جدول `Users` ثم أضفنا بعض المستخدمين. الخطوة الأهم هي إنشاء فهرس على عمود `Email`. هذا يعني أنه عندما نقوم بالبحث عن المستخدمين باستخدام بريدهم الإلكتروني، كما في الاستعلام الأخير، فإن قاعدة البيانات ستستخدم هذا الفهرس للعثور على النتائج بسرعة فائقة، بدلاً من فحص كل صف على حدة.

- الفهرسة تسرع عمليات البحث والاستعلام بشكل كبير في قواعد البيانات.
- تستخدم الفهارس بنية بيانات خاصة لتحديد موقع البيانات بسرعة.
- تساعد الفهارس على تحسين أداء التطبيقات التي تعتمد على استرجاع البيانات بكثرة.

بروتوكولات الشبكات الأساسية (HTTP / TCP/IP)

بروتوكولات الشبكات الأساسية (HTTP / TCP/IP)

لفهم كيفية بناء الأنظمة الموزعة، من الضروري فهم كيفية تواصل هذه الأنظمة مع بعضها البعض. تلعب بروتوكولات الشبكات دوراً محورياً في هذا التواصل، حيث توفر القواعد اللازمة لتبادل البيانات بكفاءة وموثوقية.

ما هي بروتوكولات الشبكات؟

بروتوكولات الشبكات هي مجموعة من القواعد والمعايير التي تحدد كيفية تنسيق البيانات وتبادلها بين الأجهزة المتصلة بالشبكة. بروتوكول **TCP/IP** (بروتوكول التحكم في الإرسال / بروتوكول الإنترنت) هو عصب الإنترنت، حيث يوفر الأساس لضمان تسليم البيانات بشكل موثوق ومنظم. بينما **HTTP** (بروتوكول نقل النص التشعبي) هو بروتوكول طبقة التطبيق الأكثر شيوعاً، ويستخدم لتصفح الويب وتبادل المعلومات بين خوادم الويب والمتصفحات أو التطبيقات.

مثال على الكود

```
curl -v http://example.com
```

شرح المثال

يستخدم هذا الأمر البسيط `curl` لإرسال طلب **HTTP GET** إلى موقع `example.com`. يعرض الخيار `-v` تفاصيل الطلب والاستجابة، بما في ذلك رؤوس HTTP وحالة الاتصال. هذا يوضح كيف يتواصل متصفحك أو تطبيقك مع خادم الويب باستخدام بروتوكول HTTP. تحت الغطاء، يتم تقسيم هذا الطلب إلى حزم أصغر وإرسالها عبر الشبكة باستخدام بروتوكول TCP/IP، الذي يضمن وصولها بالترتيب الصحيح ودون أخطاء.

- **HTTP** هو البروتوكول الرئيسي للتواصل عبر الويب، مثل تصفح المواقع.
- **TCP/IP** هو الأساس الذي يوفر الاتصال الموثوق وتنظيم البيانات على مستوى أدنى من الشبكة.

- فهم هذه البروتوكولات ضروري لتصميم أنظمة تتفاعل بكفاءة عبر الشبكات.

نظم الرسائل (Message Queues)

نظم الرسائل (Message Queues)

تخيل أن لديك أجزاء مختلفة من نظام برمجي تحتاج للتحدث مع بعضها البعض، ولكن ليس بالضرورة في نفس اللحظة. نظم الرسائل توفر طريقة موثوقة وغير متزامنة لهذه الأجزاء لتبادل المعلومات.

المفهوم الأساسي

نظام الرسائل (Message Queue) هو مكون برمجي يسمح للتطبيقات بإرسال واستقبال الرسائل. يعمل هذا النظام كمخزن مؤقت للرسائل: ترسل التطبيقات الرسائل إليه (تُعرف بالمنتجين)، وتستقبلها تطبيقات أخرى منه (تُعرف بالمستهلكين). الفائدة الرئيسية هي **فك الارتباط** بين المرسل والمستقبل، مما يعني أنهما لا يحتاجان للعمل في نفس الوقت ولا يحتاجان لمعرفة تفاصيل بعضهما البعض، مما يزيد من مرونة وقابلية تطوير النظام.

مثال برمجي

```
# مثال بسيط جداً لمفهوم طابور الرسائل (باستخدام قائمة بايثون كمحاكاة)
class MessageQueue:
    def __init__(self):
        self.queue = [] # قائمة بسيطة لتخزين الرسائل

    def send_message(self, message):
        self.queue.append(message)
        print(f"تم إرسال الرسالة: '{message}'")

    def receive_message(self):
        if self.queue:
            message = self.queue.pop(0) # استلام الرسالة الأقدم (FIFO)
            print(f"تم استلام الرسالة: '{message}'")
            return message
        else:
            print("لا توجد رسائل في الطابور")
            return None
```

الاستخدام العملي


```

print("--- بدء المحاكاة ---")
mq = MessageQueue()
mq.send_message("طلب معالجة صورة رقم 123")
mq.send_message("إشعار مستخدم جديد: أحمد")
mq.receive_message() # يستهلك الرسالة الأولى
mq.send_message("تحديث قاعدة البيانات")
mq.receive_message() # يستهلك الرسالة الثانية
mq.receive_message() # يستهلك الرسالة الثالثة
mq.receive_message() # محاولة استلام من طابور فارغ
print("--- نهاية المحاكاة ---")

```

شرح المثال

يوضح هذا المثال المبسط كيف يعمل طابور الرسائل باستخدام قائمة بايثون. عند إرسال رسالة، تضاف إلى نهاية القائمة. وعند استلام رسالة، تُزال الرسالة الأقدم من بداية القائمة (مبدأ "من يدخل أولاً يخرج أولاً" - FIFO). هذا يحاكي كيفية تخزين نظام الرسائل للمهام أو البيانات وتوزيعها على المستهلكين بشكل منظم، حتى لو لم يكن المستهلك متاحًا فورًا عند إرسال الرسالة.

- **فك الارتباط:** تسمح نظم الرسائل للمكونات بالتواصل دون الحاجة إلى أن تكون متصلة مباشرة في نفس الوقت.
- **الموثوقية:** تخزن الرسائل حتى يتم معالجتها بنجاح، مما يقلل من فقدان البيانات ويضمن إيصالها.
- **قابلية التوسع:** تسهل إضافة المزيد من المنتجين أو المستهلكين للتعامل مع أعباء العمل المتغيرة بكفاءة.

موازنة الحمل (Load Balancing)

موازنة الحمل (Load Balancing)

عندما يستقبل تطبيقك عددًا كبيرًا من المستخدمين في وقت واحد، قد يصبح الخادم الواحد نقطة اختناق. هنا يأتي دور موازنة الحمل كحل أساسي لضمان استقرار وأداء نظامك، وذلك بتوزيع عبء العمل بكفاءة.

المفهوم الأساسي

موازنة الحمل هي عملية توزيع حركة مرور الشبكة الواردة بالتساوي عبر مجموعة من الخوادم، المعروفة باسم "مجموعة الخوادم" (server farm) أو "مجمع الخوادم" (server pool). الهدف الرئيسي هو تحسين استخدام الموارد، زيادة أقصى إنتاجية، تقليل وقت الاستجابة، وتجنب التحميل الزائد على أي خادم واحد. يعمل موازن الحمل كوكيل عكسي، حيث يجلس أمام مجموعة الخوادم ويوجه طلبات العميل إلى الخادم الأكثر ملاءمة بناءً على خوارزميات محددة.

مثال برمجي

```
http {
    upstream backend_servers {
        server 192.168.1.10:8000;
        server 192.168.1.11:8000;
        server 192.168.1.12:8000;
    }

    server {
        listen 80;
        server_name example.com;

        location / {
            proxy_pass http://backend_servers;
        }
    }
}
```

شرح المثال

في هذا المثال، نستخدم Nginx كموازن حمل. يقوم الكود بتعريف مجموعة من الخوادم الخلفية (backend_servers) ضمن كتلة `upstream`، حيث يمثل كل سطر خادمًا منفصلًا بعنوان IP ومنفذ. عندما يأتي طلب إلى الخادم الرئيسي (الذي يستمع على المنفذ 80 لـ example.com)، يقوم التوجيه `location /` بتحويل الطلب إلى مجموعة الخوادم الخلفية باستخدام `proxy_pass` `http://backend_servers`. سيقوم Nginx تلقائيًا بتوزيع هذه الطلبات على الخوادم المحددة باستخدام خوارزمية افتراضية (عادةً Round Robin)، مما يضمن عدم تحمل خادم واحد للعبء بأكمله.

- **تحسين الأداء:** يوزع الطلبات لتقليل زمن الاستجابة وزيادة الإنتاجية الكلية للنظام.
- **زيادة الموثوقية:** يوفر التوفر العالي من خلال توجيه حركة المرور بعيدًا عن الخوادم الفاشلة أو التي تعمل ببطء.
- **قابلية التوسع:** يسمح بإضافة أو إزالة الخوادم بسهولة من المجمع حسب الحاجة لتلبية الطلب المتغير.

المصادقة والتفويض (Authentication & Authorization)

المصادقة والتفويض (Authentication & Authorization)

تعتبر المصادقة والتفويض من الركائز الأساسية لأمان أي نظام برمجي. تضمن هذه العمليات أن المستخدمين هم من يدعون أنهم، وأنهم لا يستطيعون الوصول إلا إلى الموارد التي يُسمح لهم بها. فهم هذين المفهومين ضروري لتصميم أنظمة قوية وآمنة.

المفهوم الأساسي

المصادقة (Authentication) هي عملية التحقق من هوية المستخدم. على سبيل المثال، عندما تقوم بتسجيل الدخول باستخدام اسم مستخدم وكلمة مرور، فإن النظام يقوم بالمصادقة عليك للتأكد من أنك الشخص الذي تدعيه. أما **التفويض (Authorization)** فهو تحديد ما يمكن للمستخدم المصادق عليه فعله أو الوصول إليه. بعد المصادقة، يقرر النظام الصلاحيات الممنوحة لك (مثلاً، هل يمكنك عرض البيانات فقط أم يمكنك تعديلها وحذفها أيضاً؟).

مثال برمجي

```
// Express.js مثال بسيط للمصادقة والتفويض باستخدام
// (token) للتحقق من التوكن middleware (Authentication) - المصادقة
const authenticateUser = (req, res, next) => {
  const token = req.headers['authorization'];
  if (!token) return res.status(401).send('لا يوجد توكن');

  // JWT (مثلاً) في نظام حقيقي، يتم التحقق من صحة التوكن هنا
  // req.user إذا كان التوكن صالحاً، نضع بيانات المستخدم في
  // مثال: مستخدم مُصادق عليه كمدیر // req.user = { id: 1, role: 'admin' };
  next();
};

// (role) للتحقق من الدور middleware (Authorization) - التفويض
```

```
const authorizeAdmin = (req, res, next) => {
  if (req.user && req.user.role === 'admin') {
    next(); // المستخدم لديه صلاحية المدير، اسمح بالوصول
  } else {
    res.status(403).send('ممنوع - ليس لديك صلاحية المدير');
  }
};

// API كيفية استخدامها في مسارات
// app.get('/profile', authenticateUser, (req, res) => { /* ... */ });
// app.post('/admin/data', authenticateUser, authorizeAdmin, (req, res) => {
```

الشرح

في هذا المثال، لدينا دالتان وسيطتان (middleware) في إطار عمل Express.js. تقوم دالة `authenticateUser` بالمصادقة عن طريق التحقق من وجود توكن في رأس الطلب. في نظام حقيقي، سيتم التحقق من صلاحية هذا التوكن لتأكيد هوية المستخدم. بعد المصادقة الناجحة، يتم إضافة معلومات المستخدم إلى الكائن `req.user`. ثم تأتي دالة `authorizeAdmin` التي تقوم بالتفويض؛ فهي تتحقق مما إذا كان المستخدم المصادق عليه لديه دور "admin" قبل السماح له بالوصول إلى مورد معين. هذا يوضح كيف تعمل العمليتان معاً لفرض الأمان.

- **المصادقة:** هي عملية التحقق من هوية المستخدم ("من أنت؟").
- **التفويض:** هي عملية تحديد صلاحيات المستخدم بعد المصادقة ("ماذا يمكنك أن تفعل؟").
- يجب أن يعمل كلا المفهومين معاً لضمان نظام آمن يمنع الوصول غير المصرح به ويحد من الأذونات.

مقاييس أداء النظام (Latency / Throughput / RPS)

مقاييس أداء النظام (Latency / Throughput / RPS)

يعد فهم كيفية قياس أداء نظامك أمرًا بالغ الأهمية لضمان عمله بكفاءة وفعالية. تساعدنا هذه المقاييس على تحديد الاختناقات وتحسين تجربة المستخدم.

ما هي Latency, Throughput, RPS؟

Latency (زمن الاستجابة): هو الوقت الذي يستغرقه النظام للاستجابة لطلب واحد. بعبارة أخرى، هو التأخير بين إرسال الطلب وتلقي الاستجابة. كلما كان زمن الاستجابة أقل، كان النظام أسرع وأفضل.

Throughput (الإنتاجية): تشير إلى حجم العمل (مثل عدد العمليات أو كمية البيانات) التي يمكن للنظام معالجتها بنجاح خلال فترة زمنية محددة. كلما كانت الإنتاجية أعلى، كان النظام أكثر قدرة على التعامل مع الأعباء.

RPS (الطلبات في الثانية - Requests Per Second): هو مقياس محدد للإنتاجية يوضح عدد الطلبات التي يمكن للنظام معالجتها وخدمتها في ثانية واحدة. يستخدم بشكل شائع في أنظمة الويب والخدمات لتحديد قدرة الخادم.

مثال برمجي

```
import time

def simulate_request():
    # simulates work being done, e.g., database query or computation
    time.sleep(0.05) # Simulate a 50ms operation

num_requests = 100 # Total number of requests to simulate

start_time = time.perf_counter()
latencies = []

for _ in range(num_requests):
    request_start = time.perf_counter()
```

```

simulate_request()
request_end = time.perf_counter()
latencies.append(request_end - request_start)

end_time = time.perf_counter()

total_time_taken = end_time - start_time

# Calculate metrics
avg_latency = (sum(latencies) / len(latencies)) * 1000 # in milliseconds
rps = num_requests / total_time_taken

print(f"Total Requests: {num_requests}")
print(f"Total Time Taken: {total_time_taken:.2f} seconds")
print(f"Average Latency: {avg_latency:.2f} ms")
print(f"Requests Per Second (RPS): {rps:.2f}")

```

شرح المثال

يُظهر هذا الكود كيفية محاكاة قياس مقاييس الأداء الأساسية. نقوم بمحاكاة 100 طلب، حيث يستغرق كل طلب 50 ميلي ثانية (0.05 ثانية) لإنجازه. يتم قياس زمن كل طلب على حدة لحساب متوسط زمن الاستجابة (Latency)، ويتم حساب إجمالي الوقت المستغرق لمعالجة جميع الطلبات لحساب عدد الطلبات في الثانية (RPS).

- **Latency**: هو الوقت المستغرق للاستجابة لطلب واحد، وكلما قل كان أفضل.
- **Throughput/RPS**: يمثلان كمية العمل التي يمكن للنظام إنجازها في فترة زمنية معينة، وكلما زادت كان أفضل.
- فهم هذه المقاييس وتحسينها ضروري لبناء أنظمة قوية وفعالة تستجيب لمتطلبات المستخدمين.

مراقبة الأنظمة والتسجيل (Monitoring & Logging)

مراقبة الأنظمة والتسجيل (Monitoring & Logging)

في تصميم الأنظمة، تعتبر مراقبة الأداء وتسجيل الأحداث أدوات حيوية لضمان استقرار النظام وأمانه. تتيح لنا هذه الممارسات فهم كيفية عمل أنظمتنا وكشف المشكلات مبكراً.

المفهوم الأساسي: المراقبة والتسجيل

المراقبة (Monitoring) هي عملية جمع وتحليل البيانات بشكل مستمر حول حالة موارد النظام وأدائه، مثل استخدام وحدة المعالجة المركزية (CPU)، الذاكرة، حركة مرور الشبكة، ووقت الاستجابة. بينما **التسجيل (Logging)** هو حفظ سجلات للأحداث التي تحدث داخل النظام، مثل محاولات تسجيل الدخول، الأخطاء، والمعاملات، في ملفات أو قواعد بيانات خاصة. معاً، يوفران رؤية شاملة لأداء النظام وصحته.

مثال برمجي

```
import logging
import datetime

# تهيئة المسجل (Logger)
logging.basicConfig(filename='app.log', level=logging.INFO,
                    format='%(asctime)s - %(levelname)s - %(message)s')

def process_request(request_id):
    """
    دالة تحاكي معالجة طلب وتسجيل الأحداث.
    """
    logging.info(f"[{datetime.datetime.now()}] بدء معالجة الطلب {request_id}")
    try:
        # محاكاة بعض العمل
        if request_id % 2 == 0:
            raise ValueError("معرف الطلب الزوجي يسبب خطأ")
        logging.debug(f"الطلب {request_id} تم معالجته بنجاح")
    except Exception as e:
        logging.error(f"حدث خطأ أثناء معالجة الطلب {request_id}: {e}")
    logging.info(f"[{datetime.datetime.now()}] انتهاء معالجة الطلب {request_id}")
```



```
# اختبار الدالة
process_request(1)
process_request(2)
process_request(3)

print("app.log تم تسجيل الأحداث في")
```

شرح المثال

يُظهر هذا المثال كيفية استخدام وحدة التسجيل (`logging`) في بايثون لإنشاء سجلات للأحداث. قمنا بتهيئة المسجل لكتابة الرسائل إلى ملف `app.log` بمستوى معلوماتي (`INFO`) أو أعلى. تتضمن الرسائل وقت الحدث، مستواه (مثل `ERROR` ، `INFO`)، والرسالة نفسها. هذا يساعد المطورين والمهندسين على تتبع تدفق التطبيق وتحديد المشاكل بسرعة.

- تساعد المراقبة في فهم أداء النظام وتحديد الاختناقات المحتملة.
- التسجيل ضروري لتتبع الأحداث، استكشاف الأخطاء وإصلاحها، وتوفير أدلة للأمان.
- الجمع بين المراقبة والتسجيل يوفر رؤية شاملة لصحة النظام واستقراره.